

DATA ENGINEERING INTERN

The *practical* test

Write a Playwright script that pulls structured data off a live Nepali news site. This mirrors the work you would actually do here: read a page, find the selectors, get clean JSON out.

90 MINUTES
EXPECTED

2 EXTRACTION
TASKS

5 FILES TO
SUBMIT

POSITION	Data Engineering Intern, Dataweav
DURATION	60–90 minutes
TARGET SITE	ekantipur.com
SUBMISSION	Through the Google Form linked in your invitation email: one zip of your project, and the recording as a separate upload. See the end of this document.

READ THIS FIRST

All five files, or *automatic rejection*.

We do not chase missing deliverables. A submission short of any one of the five files is rejected without review, regardless of how good the rest of it is.

01

Overview

This practical test evaluates your ability to write browser automation scripts for web scraping. You will create a Playwright script in Python that extracts structured data from a Nepali news website. This assessment mirrors real tasks you will encounter at Dataweav, where we build enterprise-grade data extraction solutions.

Note: Use whichever AI coding assistant you like. We expect you to use one. The Appendix explains how to get one for free and, more importantly, how we expect you to work with it: inspect the page yourself, instruct it with what you found, review what it gives back, and verify the result against the live site.

02

Task description

Target website: <https://ekantipur.com>

Create a Playwright automation script in Python that performs the following:

Task 1 — Extract entertainment news

Navigate to the Entertainment (मनोरञ्जन) section and extract the **top 5 news articles** with the following data points:

FIELD	DESCRIPTION
<code>title</code>	The headline/title of the news article
<code>image_url</code>	URL of the post/thumbnail image (full URL)
<code>category</code>	The category/section label (e.g. मनोरञ्जन, बलिउड)
<code>author</code>	Author name (if available; use <code>null</code> if not found)

Task 2 — Extract cartoon of the day

Locate and extract the "Cartoon of the Day" (व्यंग्यचित्र) section with:

FIELD	DESCRIPTION
<code>title</code>	The cartoon's title or caption
<code>image_url</code>	Direct URL to the cartoon image
<code>author</code>	The cartoonist's name

03

Expected output format

Save your extracted data as a JSON file named `output.json` with the following structure:

OUTPUT.JSON

```
{
  "entertainment_news": [
    {
      "title": "Article headline here",
      "image_url": "https://...",
      "category": "मनोरञ्जन",
      "author": "Author Name"
    },
    {
      "title": "Second article headline",
      "image_url": "https://...",
      "category": "बलिउड",
      "author": null
    }
  ],
  "cartoon_of_the_day": {
    "title": "Cartoon title",
    "image_url": "https://...",
    "author": "Cartoonist Name"
  }
}
```

04

Technical requirements

- Use **Python 3.11+** with **Playwright**
- Use **uv** as the package manager
- Handle dynamic content loading (wait for elements properly)
- Implement error handling for missing elements
- Use appropriate selectors: prefer data attributes, then semantic HTML, avoid fragile XPaths
- Add meaningful comments explaining your approach
- Output must be valid JSON saved to `output.json`

05

Setup instructions

Step 1 — Install uv (package manager)

MACOS / LINUX

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

WINDOWS (POWERSHELL)

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

After installation, restart your terminal or run:

```
source $HOME/.local/bin/env # Linux/macOS
```

Step 2 — Create the project

```
# Create project directory
mkdir ekantipur-scraper
cd ekantipur-scraper

# Initialize Python project with uv
uv init

# Add dependencies
uv add playwright

# Install Playwright browsers
uv run playwright install chromium
```

Step 3 — Verify installation

```
# Check Python version
uv run python --version

# Verify Playwright is installed
uv run python -c "from playwright.sync_api import sync_playwright; print('Playwright OK')"
```

Project structure

Your project should look like this:

```
ekantipur-scraper/
├─ pyproject.toml    # Project config (auto-generated by uv)
├─ uv.lock           # Lock file (auto-generated)
├─ scraper.py        # Your main script
└─ output.json       # Generated output
```

Running your script

```
uv run python scraper.py
```

Helpful Playwright methods

Here are some Playwright methods you will likely need:

```
# Navigation
page.goto("https://example.com")
page.click("a.nav-link")

# Waiting for elements
page.wait_for_selector(".article-card")
page.wait_for_load_state("networkidle")

# Selecting elements
elements = page.query_selector_all(".news-item") # Multiple elements
element  = page.query_selector(".single-item")   # Single element

# Extracting data from single element
text = element.text_content()
href = element.get_attribute("href")
src  = element.query_selector("img").get_attribute("src")

# Looping through elements
cards = page.query_selector_all(".news-card")
for card in cards:
    title = card.query_selector("h2").text_content()
    image = card.query_selector("img").get_attribute("src")

# Handling missing elements safely
author_el  = card.query_selector(".author")
author_name = author_el.text_content() if author_el else None

# Using locators (recommended approach)
page.locator(".news-card").first.click()
titles = page.locator(".news-card h2").all_text_contents()
```

Tips for success

01

Inspect the website first

Use browser DevTools (F12) to understand the HTML structure before writing code.

03

Use headless=False

Watch the browser to debug issues visually.

05

Check for lazy loading

Some images load dynamically. You may need to scroll or wait.

07

Use try/except blocks

Wrap element access in error handling for robustness.

02

Start simple

Get one piece of data working before extracting everything.

04

Handle Nepali text

The site uses Devanagari script. Use `ensure_ascii=False` when writing JSON.

06

Test incrementally

Use `print()` statements to verify each extraction step.

08

No hardcoded data

Every value in `output.json` must come from the live page.

Your working notes

Submit a `notes.md` with three headings. Keep it short: half a page is enough. We read this before we read your code, because it tells us more.

1. Selectors I found

The notes you took in DevTools before writing any code: which elements hold the title, image, category and author, and anything unusual you noticed about how the page loads.

2. What I asked the AI

Paste the actual instructions you gave, verbatim. Not a summary, and not a tidied-up version. If you went back and forth, show the ones that mattered.

3. One thing it got wrong

Something your AI assistant produced that was incorrect, and how you caught it. A wrong selector, a placeholder image URL, a crash on a missing author, an invented attribute. Say what it did, how you noticed, and what you changed.

The third heading is the one we weigh most heavily. If you genuinely hit nothing wrong, say that and describe how you checked, but understand that we will test the claim in the interview.

09

Submission checklist

Before submitting, verify:

- ☐ Script runs without errors using `uv run python scraper.py`
- ☐ `output.json` contains valid JSON with proper encoding
- ☐ All 5 entertainment articles are extracted with complete data
- ☐ Cartoon of the Day data is complete
- ☐ Code has meaningful comments
- ☐ No hardcoded data, all data comes from scraping
- ☐ `notes.md` has all three headings filled in
- ☐ Recording says which AI tool and model you used
- ☐ Zip opens to reveal all four files, with no `.venv` folder inside
- ☐ Zip and recording both uploaded to the Google Form, and you saw the confirmation screen

WHAT TO SEND

Five files. *All of them.*

- 01 `scraper.py` — your main script
- 02 `output.json` — extracted data
- 03 `pyproject.toml` — project dependencies
- 04 `notes.md` — your working notes, in the three parts set out in section 08
- 05 `recording.mp4` — screen recording with voiceover explaining how you found the selectors, which AI tool you used and how, and the overall code flow

How to package it: zip the first four into a single `.zip` and upload that. Upload the recording separately, it does not belong in the zip. Use `.zip`, not `.rar`, and leave the `.venv` folder out.

Submit through the Google Form linked in your invitation email.

You must be signed in to a Google account to upload.

If the recording is too big to upload, the form takes a Drive or YouTube link instead.

Missing any one file means automatic rejection.